

Verifiable Inference

Settlement is a function of elapsed time alone. Which model answered is a field in the provider's local config that no counterparty reads and no contract records.

What the contract measures. Provider revenue is `pricePerSecond` times session duration (`IBidStorage.sol:14`, `SessionRouter.sol:374`). Nothing in that product refers to tokens produced, weights loaded, or the machine that answered. The map from an on-chain `modelId` to the upstream that serves it is a config field (`aiengine/ai_engine.go:64`). So serving a 7B where a 70B was advertised is not an attack on the protocol but a configuration choice that keeps the whole cost difference and leaves no artifact. Per-second pricing makes the incentive continuous and unmetered: adverse selection in textbook form.

Three questions that get merged. *Is the box genuine?* You answer this, and answer it well: 5,439 lines binding TDX `report_data` to a TLS SPKI digest, with the NVIDIA `eat_nonce` compared in constant time. One caveat. `VerifyQuote` (`verifier.go:565`) POSTs the quote to a third party's `/api/quote-parse` and infers validity from the JSON it gets back, and the measurement registry is fetched unsigned from a CDN mirror because the upstream repository "was removed" (`artifacts_registry.go:18`). The one question you answer rests on a rented root of trust whose artifacts already 404. *Did inference happen at all?* Nothing answers this. *Are these the advertised weights?* HideNSeek attempted it; last commit 2024-07-29, `main.py` is zero bytes.

Mechanism, cheapest layer first. *Commit.* Add `bytes32 modelCommitment` to `Bid`: the hash of a canonical manifest fixing weight digests, quantization, tokenizer and serving config. One `SSTORE` per bid, none per request. A zero commitment stays legal and pledges nothing, which honestly describes every bid today. It detects nothing on its own; it makes substitution decidable, which is what the other two layers need. *Bind.* The 64 `report_data` bytes are fully allocated, so change what the GPU nonce *is* rather than the layout: `gpuNonce = SHA256("mor-gpu-nonce-v1" || c ||  $\rho$ )`, with fresh ρ published beside the quote, so freshness holds, the CPU-GPU check is untouched, and the Intel-signed quote now covers c . Per response, the enclave's TLS key signs (`sessionId, i, c, H(x), H(y)`) and the Merkle root of those leaves rides in the close-out receipt. Without a TEE it degrades to a plain `secp256k1` signature over the same statement, which is still adjudicable. *Probe.* Sample sessions with known-answer challenges, slash on contradiction.

Sizing the probe budget. Let γ be what substitution saves as a fraction of epoch revenue, λ the slashed fraction of stake, β the chance one probe catches it. Expected penalty exceeds expected saving when $m \geq \beta^{-1} \ln(\lambda/(\lambda - \gamma))$ probes per provider per epoch; at $\gamma = 0.9$, $\lambda = 1$, $\beta = 0.3$ that is eight. The bound carries no traffic term: the duty cycle falls as 1/volume, so the scheme gets cheaper as the market grows. It also requires $\lambda > \gamma$: stake smaller than the prize deters nothing at any probe rate. That is affordable because the bond exists already: `_claimForProvider` clamps epoch earnings to `provider.stake - provider.limitPeriodEarned` (`SessionRouter.sol:383`), so revenue and stake are denominated the same way.

What Morpheus gets

- A claim consumers can filter on, and a floor under price competition against providers who do not run what they advertise.
- `Provider.stake` becomes a bond rather than a withdrawal delay, and attestation starts contributing to the payment decision.
- A specification implementable without adopting Lux or Hanzo software.

What we get, stated plainly

- A market whose model claims are checkable, which is our precondition for routing production traffic into it.
- We contribute the manifest encoder, verifier reference and probe harness, plus ML-DSA precompiles for receipts that outlive `secp256k1`.

First step, and who takes it. Whoever owns the Marketplace facet opens a PR adding `modelCommitment` to `Bid` and `postModelBid`, emitted in `MarketplaceBidPosted` (`IMarketplace.sol:8`). A facet replacement is what EIP-2535 is for: one 5-of-9 Safe transaction, no change to settlement arithmetic, no open session at risk. Nobody has to verify anything on day one; consumers can filter from the first event.

What this does not solve. None of it proves the arithmetic was right. An enclave that loads the pledged weights and then truncates the attention window still produces a valid receipt, and proving inference in zero knowledge is out of reach at these model sizes.